

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт по дисциплине
Вычислительная математика
Лабораторная работа №4
«Дискретное преобразование Фурье»
Вариант 18

Выполнил:
студент группы 5130201/40003
Джабраилов А.В.
Подпись: _____

Принял:
К. физ.-мат. наук, доц., доц.
Пак В.Г.
Подпись: _____
«_____» _____ 2026 г.

Санкт-Петербург, 2026

1 Задание

1. Применяя дискретное преобразование Фурье для данного набора сигналов, получить вектор амплитуд, затем по нему обратным преобразованием получить исходный сигнал.
2. Применяя быстрое преобразование Фурье с прореживанием по времени по основанию 2 для данного набора сигналов, получить векторы амплитуд по чётным и нечётным отсчётам, вектор амплитуд по всем отсчётам; затем по нему обратным преобразованием получить векторы обратных преобразований по чётным и нечётным отсчётам и исходный сигнал.

2 Цель работы

Освоить алгоритмы дискретного и быстрого преобразований Фурье, реализовать прямое и обратное преобразования для заданного дискретного сигнала и проверить восстановление исходных отсчётов.

3 Ход работы

Исходные данные варианта 18 содержат $n = 16$ временных отсчётов.

Таблица 1: Таблица исходных отсчётов.

j	0	1	2	3	4	5	6	7
y_j	-0,78	-1,22	-1,34	-1,39	-1,42	-1,43	-1,42	-1,41

j	8	9	10	11	12	13	14	15
y_j	-1,37	-1,30	-1,10	-0,10	1,12	1,25	1,33	1,36

Алгоритм вычислений:

1. Вычислить вектор комплексных амплитуд F_k по формуле прямого дискретного преобразования Фурье.
2. По найденному вектору F_k выполнить обратное дискретное преобразование Фурье и сравнить результат с исходным сигналом.
3. Разделить исходный вектор на отсчёты с чётными и нечётными индексами.
4. Для полученных подвекторов вычислить векторы амплитуд F_k^0 и F_k^1 .
5. По формулам быстрого преобразования Фурье получить общий вектор амплитуд F_k .
6. Выполнить обратное БПФ по формуле (4), получить вспомогательные векторы Y_j^0 , Y_j^1 и восстановить исходный сигнал.

Для реализации использован язык *Python*. В программе все преобразования вычисляются явно по формулам методических указаний, без использования готовой функции БПФ.

3.1 Дискретное преобразование Фурье

Прямое дискретное преобразование Фурье определяется формулой:

$$F_k = \sum_{j=0}^{n-1} y_j \exp \left(-\frac{2\pi i k}{n} j \right), \quad k = 0, 1, \dots, n-1.$$

Обратное преобразование определяется формулой:

$$y_j = \frac{1}{n} \sum_{k=0}^{n-1} F_k \exp \left(\frac{2\pi i k}{n} j \right), \quad j = 0, 1, \dots, n-1.$$

Подставим $n = 16$ и значения исходных отсчётов в формулу прямого ДПФ:

$$\begin{aligned} F_k = & -0,78 \exp \left(-\frac{2\pi i k}{16} \cdot 0 \right) - 1,22 \exp \left(-\frac{2\pi i k}{16} \cdot 1 \right) - 1,34 \exp \left(-\frac{2\pi i k}{16} \cdot 2 \right) \\ & - 1,39 \exp \left(-\frac{2\pi i k}{16} \cdot 3 \right) - 1,42 \exp \left(-\frac{2\pi i k}{16} \cdot 4 \right) - 1,43 \exp \left(-\frac{2\pi i k}{16} \cdot 5 \right) \\ & - 1,42 \exp \left(-\frac{2\pi i k}{16} \cdot 6 \right) - 1,41 \exp \left(-\frac{2\pi i k}{16} \cdot 7 \right) - 1,37 \exp \left(-\frac{2\pi i k}{16} \cdot 8 \right) \\ & - 1,30 \exp \left(-\frac{2\pi i k}{16} \cdot 9 \right) - 1,10 \exp \left(-\frac{2\pi i k}{16} \cdot 10 \right) - 0,10 \exp \left(-\frac{2\pi i k}{16} \cdot 11 \right) \\ & + 1,12 \exp \left(-\frac{2\pi i k}{16} \cdot 12 \right) + 1,25 \exp \left(-\frac{2\pi i k}{16} \cdot 13 \right) \\ & + 1,33 \exp \left(-\frac{2\pi i k}{16} \cdot 14 \right) + 1,36 \exp \left(-\frac{2\pi i k}{16} \cdot 15 \right). \end{aligned}$$

Например, для первых двух амплитуд:

$$\begin{aligned} F_0 = & -0,78 - 1,22 - 1,34 - 1,39 - 1,42 - 1,43 - 1,42 - 1,41 \\ & - 1,37 - 1,30 - 1,10 - 0,10 + 1,12 + 1,25 + 1,33 + 1,36 = -9,220000, \\ F_1 \approx & 5,529825 + 9,351469i. \end{aligned}$$

Подставим найденные амплитуды в формулу обратного ДПФ:

$$\begin{aligned} \hat{y}_j = & \frac{1}{16} \left(-9,220000 \exp \left(\frac{2\pi i \cdot 0}{16} j \right) + (5,529825 + 9,351469i) \exp \left(\frac{2\pi i \cdot 1}{16} j \right) \right. \\ & + (-2,486396 + 5,022864i) \exp \left(\frac{2\pi i \cdot 2}{16} j \right) + (-1,378383 + 0,540232i) \exp \left(\frac{2\pi i \cdot 3}{16} j \right) \\ & + (0,080000 + 1,160000i) \exp \left(\frac{2\pi i \cdot 4}{16} j \right) + (-0,991293 + 1,391733i) \exp \left(\frac{2\pi i \cdot 5}{16} j \right) \\ & + (-1,213604 + 0,322864i) \exp \left(\frac{2\pi i \cdot 6}{16} j \right) + (-0,800149 + 0,042971i) \exp \left(\frac{2\pi i \cdot 7}{16} j \right) \\ & - 0,740000 \exp \left(\frac{2\pi i \cdot 8}{16} j \right) + (-0,800149 - 0,042971i) \exp \left(\frac{2\pi i \cdot 9}{16} j \right) \\ & + (-1,213604 - 0,322864i) \exp \left(\frac{2\pi i \cdot 10}{16} j \right) + (-0,991293 - 1,391733i) \exp \left(\frac{2\pi i \cdot 11}{16} j \right) \\ & + (0,080000 - 1,160000i) \exp \left(\frac{2\pi i \cdot 12}{16} j \right) + (-1,378383 - 0,540232i) \exp \left(\frac{2\pi i \cdot 13}{16} j \right) \\ & \left. + (-2,486396 - 5,022864i) \exp \left(\frac{2\pi i \cdot 14}{16} j \right) + (5,529825 - 9,351469i) \exp \left(\frac{2\pi i \cdot 15}{16} j \right) \right). \end{aligned}$$

3.2 Быстрое преобразование Фурье

Вектор отсчётов разделяется на два подвектора половинной длины:

$$y_j^0 = y_{2j}, \quad y_j^1 = y_{2j+1}, \quad j = 0, 1, \dots, \frac{n}{2} - 1.$$

Для варианта 18:

$$\bar{y}^0 = (-0,78, -1,34, -1,42, -1,42, -1,37, -1,10, 1,12, 1,33),$$

$$\bar{y}^1 = (-1,22, -1,39, -1,43, -1,41, -1,30, -0,10, 1,25, 1,36).$$

Введём обозначение из методических указаний:

$$W_n^{j \cdot k} = \exp \left(-\frac{2\pi i k}{n} j \right).$$

Тогда для подвекторов:

$$F_k^0 = \sum_{j=0}^{\frac{n}{2}-1} y_{2j} W_{\frac{n}{2}}^{j \cdot k}, \quad F_k^1 = \sum_{j=0}^{\frac{n}{2}-1} y_{2j+1} W_{\frac{n}{2}}^{j \cdot k}.$$

Подставим $n = 16$, то есть для подвекторов используется $W_8^{j \cdot k}$:

$$\begin{aligned} F_k^0 &= -0,78 \exp \left(-\frac{2\pi i k}{8} \cdot 0 \right) - 1,34 \exp \left(-\frac{2\pi i k}{8} \cdot 1 \right) - 1,42 \exp \left(-\frac{2\pi i k}{8} \cdot 2 \right) \\ &\quad - 1,42 \exp \left(-\frac{2\pi i k}{8} \cdot 3 \right) - 1,37 \exp \left(-\frac{2\pi i k}{8} \cdot 4 \right) - 1,10 \exp \left(-\frac{2\pi i k}{8} \cdot 5 \right) \\ &\quad + 1,12 \exp \left(-\frac{2\pi i k}{8} \cdot 6 \right) + 1,33 \exp \left(-\frac{2\pi i k}{8} \cdot 7 \right), \\ F_k^1 &= -1,22 \exp \left(-\frac{2\pi i k}{8} \cdot 0 \right) - 1,39 \exp \left(-\frac{2\pi i k}{8} \cdot 1 \right) - 1,43 \exp \left(-\frac{2\pi i k}{8} \cdot 2 \right) \\ &\quad - 1,41 \exp \left(-\frac{2\pi i k}{8} \cdot 3 \right) - 1,30 \exp \left(-\frac{2\pi i k}{8} \cdot 4 \right) - 0,10 \exp \left(-\frac{2\pi i k}{8} \cdot 5 \right) \\ &\quad + 1,25 \exp \left(-\frac{2\pi i k}{8} \cdot 6 \right) + 1,36 \exp \left(-\frac{2\pi i k}{8} \cdot 7 \right). \end{aligned}$$

По формулам БПФ:

$$\begin{cases} F_k = F_k^0 + W_n^k F_k^1, \\ F_{k+\frac{n}{2}} = F_k^0 - W_n^k F_k^1, \end{cases} \quad k = 0, \dots, \frac{n}{2} - 1.$$

Например, для $k = 1$:

$$\begin{aligned} F_1 &= (2,364838 + 4,654249i) + \exp \left(-\frac{2\pi i}{16} \right) (1,126518 + 5,550854i) \\ &= 5,529825 + 9,351469i, \\ F_9 &= (2,364838 + 4,654249i) - \exp \left(-\frac{2\pi i}{16} \right) (1,126518 + 5,550854i) \\ &= -0,800149 - 0,042971i. \end{aligned}$$

Обратное БПФ производится по формулам из методических указаний:

$$\begin{cases} y_j = Y_j^0 + W_n^{-j} Y_j^1, \\ y_{j+\frac{n}{2}} = Y_j^0 - W_n^{-j} Y_j^1, \end{cases} \quad j = 0, \dots, \frac{n}{2} - 1,$$

где

$$Y_j^0 = \frac{1}{n} \sum_{k=0}^{\frac{n}{2}-1} F_k^0 W_{\frac{n}{2}}^{-j \cdot k}, \quad Y_j^1 = \frac{1}{n} \sum_{k=0}^{\frac{n}{2}-1} F_k^1 W_{\frac{n}{2}}^{-j \cdot k}.$$

Для вычисления по этой формуле вектор амплитуд F разделён на компоненты с чётными и нечётными индексами:

$$\bar{F}^0 = (F_0, F_2, F_4, F_6, F_8, F_{10}, F_{12}, F_{14}), \quad \bar{F}^1 = (F_1, F_3, F_5, F_7, F_9, F_{11}, F_{13}, F_{15}).$$

Подставим $n = 16$. Например, для $j = 0$:

$$\begin{aligned} Y_0^0 &= \frac{1}{16} ((-9,220000) + (-2,486396 + 5,022864i) + (0,080000 + 1,160000i) \\ &\quad + (-1,213604 + 0,322864i) - 0,740000 + (-1,213604 - 0,322864i) \\ &\quad + (0,080000 - 1,160000i) + (-2,486396 - 5,022864i)) = -1,075000, \\ Y_0^1 &= \frac{1}{16} ((5,529825 + 9,351469i) + (-1,378383 + 0,540232i) + (-0,991293 + 1,391733i) \\ &\quad + (-0,800149 + 0,042971i) + (-0,800149 - 0,042971i) + (-0,991293 - 1,391733i) \\ &\quad + (-1,378383 - 0,540232i) + (5,529825 - 9,351469i)) = 0,295000. \end{aligned}$$

Тогда

$$y_0 = Y_0^0 + W_{16}^0 Y_0^1 = -1,075000 + 0,295000 = -0,780000,$$

$$y_8 = Y_0^0 - W_{16}^0 Y_0^1 = -1,075000 - 0,295000 = -1,370000.$$

4 Результаты

4.1 Дискретное преобразование Фурье

Вектор комплексных амплитуд, полученный по формуле прямого ДПФ:

Таблица 2: Вектор амплитуд $F(k)$.

k	$F(k)$	k	$F(k)$
0	$-9,220000$	8	$-0,740000$
1	$5,529825 + 9,351469i$	9	$-0,800149 - 0,042971i$
2	$-2,486396 + 5,022864i$	10	$-1,213604 - 0,322864i$
3	$-1,378383 + 0,540232i$	11	$-0,991293 - 1,391733i$
4	$0,080000 + 1,160000i$	12	$0,080000 - 1,160000i$
5	$-0,991293 + 1,391733i$	13	$-1,378383 - 0,540232i$
6	$-1,213604 + 0,322864i$	14	$-2,486396 - 5,022864i$
7	$-0,800149 + 0,042971i$	15	$5,529825 - 9,351469i$

Результат обратного преобразования совпадает с исходным сигналом с точностью до машинной погрешности.

Таблица 3: Проверка обратного ДПФ.

j	y_j	$\hat{y}_j$	$ y_j - \hat{y}_j $
0	$-0,780000$	$-0,780000$	$4,56 \cdot 10^{-15}$
1	$-1,220000$	$-1,220000$	$3,97 \cdot 10^{-15}$
2	$-1,340000$	$-1,340000$	$4,70 \cdot 10^{-15}$
3	$-1,390000$	$-1,390000$	$2,20 \cdot 10^{-15}$
4	$-1,420000$	$-1,420000$	$3,33 \cdot 10^{-15}$
5	$-1,430000$	$-1,430000$	$3,71 \cdot 10^{-15}$
6	$-1,420000$	$-1,420000$	$2,18 \cdot 10^{-15}$
7	$-1,410000$	$-1,410000$	$1,82 \cdot 10^{-15}$
8	$-1,370000$	$-1,370000$	$1,11 \cdot 10^{-15}$
9	$-1,300000$	$-1,300000$	$4,60 \cdot 10^{-15}$
10	$-1,100000$	$-1,100000$	$3,30 \cdot 10^{-15}$
11	$-0,100000$	$-0,100000$	$2,37 \cdot 10^{-15}$
12	$1,120000$	$1,120000$	$3,34 \cdot 10^{-15}$
13	$1,250000$	$1,250000$	$2,26 \cdot 10^{-15}$
14	$1,330000$	$1,330000$	$7,34 \cdot 10^{-15}$
15	$1,360000$	$1,360000$	$1,57 \cdot 10^{-15}$

4.2 Быстрое преобразование Фурье

После разделения исходного сигнала на отсчёты с чётными и нечётными индексами получены следующие вспомогательные векторы амплитуд:

Таблица 4: Векторы амплитуд $F^0(k)$ и $F^1(k)$.

k	$F^0(k)$	$F^1(k)$
0	$-4,980000$	$-4,240000$
1	$2,364838 + 4,654249i$	$1,126518 + 5,550854i$
2	$-1,850000 + 2,350000i$	$-2,340000 + 1,440000i$
3	$-1,184838 - 0,425751i$	$-0,966518 + 0,190854i$
4	$0,080000$	$-1,160000$
5	$-1,184838 + 0,425751i$	$-0,966518 - 0,190854i$
6	$-1,850000 - 2,350000i$	$-2,340000 - 1,440000i$
7	$2,364838 - 4,654249i$	$1,126518 - 5,550854i$

По формулам БПФ получен тот же вектор амплитуд $F(k)$, что и при прямом ДПФ (таблица 2). Максимальное отличие между прямым ДПФ и БПФ составило $6,58 \cdot 10^{-14}$.

Вспомогательные векторы обратного БПФ, вычисленные по формуле (4):

Таблица 5: Векторы $Y^0(j)$ и $Y^1(j)$.

j	$Y^0(j)$	$Y^1(j)$
0	$-1,075000$	$0,295000$
1	$-1,260000$	$0,036955 - 0,015307i$
2	$-1,220000$	$-0,084853 + 0,084853i$
3	$-0,745000$	$-0,246831 + 0,595902i$
4	$-0,150000$	$1,270000i$
5	$-0,090000$	$0,512796 + 1,237999i$
6	$-0,045000$	$0,972272 + 0,972272i$
7	$-0,025000$	$1,279573 + 0,530017i$

По формуле (4) восстановлен исходный сигнал. Максимальная погрешность восстановления составила $3,78 \cdot 10^{-15}$.

Как видно из таблиц, обратные преобразования возвращают исходные отсчёты, а отклонения имеют порядок машинной точности.

5 Приложение

Listing 1: Файл main.py

```
1 import cmath
2 import math
3 import re
4 from pathlib import Path
5
6
7 ROOT = Path(__file__).resolve().parents[1]
8 VARIANT_FILE = ROOT / "task" / "var.md"
9 EPS = 1e-10
10
11
12 def parse_variant(path):
13     text = path.read_text(encoding="utf-8")
14
15     variant_match = re.search(r"variant:\s*(\d+)", text)
16     values_match = re.search(r"values:\s*(.+) ", text)
17     if not variant_match or not values_match:
18         raise ValueError("File var.md must contain 'variant:' and 'values:' lines.")
19
20     variant = int(variant_match.group(1))
21     values = [float(item.replace(",", ".")) for item in values_match.group(1).split()]
22     return variant, values
23
24
25 def dft(values):
26     n = len(values)
27     result = []
28     for k in range(n):
29         total = 0j
30         for j, value in enumerate(values):
31             total += value * cmath.exp(-2j * math.pi * k * j / n)
32         result.append(total)
33     return result
34
35
36 def inverse_dft(amplitudes):
37     n = len(amplitudes)
38     result = []
39     for j in range(n):
40         total = 0j
41         for k, value in enumerate(amplitudes):
42             total += value * cmath.exp(2j * math.pi * k * j / n)
```

```

43         result.append(total / n)
44     return result
45
46
47 def fft_decimation_by_time(values):
48     n = len(values)
49     if n % 2 != 0:
50         raise ValueError("The number of samples must be even for radix-2
51         decimation.")
52
53     even_values = values[0::2]
54     odd_values = values[1::2]
55     even_amplitudes = dft(even_values)
56     odd_amplitudes = dft(odd_values)
57
58     amplitudes = [0j] * n
59     for k in range(n // 2):
60         multiplier = cmath.exp(-2j * math.pi * k / n)
61         amplitudes[k] = even_amplitudes[k] + multiplier * odd_amplitudes[k]
62         amplitudes[k + n // 2] = even_amplitudes[k] - multiplier *
63         odd_amplitudes[k]
64
65     return even_values, odd_values, even_amplitudes, odd_amplitudes,
66     amplitudes
67
68
69 def inverse_fft_decimation_by_time(amplitudes):
70     n = len(amplitudes)
71     if n % 2 != 0:
72         raise ValueError("The number of amplitudes must be even for radix-2
73         decimation.")
74
75     even_frequency_amplitudes = amplitudes[0::2]
76     odd_frequency_amplitudes = amplitudes[1::2]
77     half = n // 2
78
79     y0 = []
80     y1 = []
81     first_half = []
82     second_half = []
83
84     for j in range(half):
85         y0_j = 0j
86         y1_j = 0j
87         for k in range(half):
88             multiplier = cmath.exp(2j * math.pi * k * j / half)
89             y0_j += even_frequency_amplitudes[k] * multiplier
90             y1_j += odd_frequency_amplitudes[k] * multiplier

```

```

87
88     y0_j /= n
89     y1_j /= n
90     y0.append(y0_j)
91     y1.append(y1_j)
92
93     butterfly_multiplier = cmath.exp(2j * math.pi * j / n)
94     first_half.append(y0_j + butterfly_multiplier * y1_j)
95     second_half.append(y0_j - butterfly_multiplier * y1_j)
96
97     return y0, y1, first_half + second_half
98
99
100 def clean_number(value, digits=6):
101     if abs(value) < EPS:
102         value = 0.0
103     return f"{value:.{digits}f}"
104
105
106 def format_complex(value, digits=6):
107     real = 0.0 if abs(value.real) < EPS else value.real
108     imag = 0.0 if abs(value.imag) < EPS else value.imag
109
110     if imag == 0.0:
111         return clean_number(real, digits)
112     if real == 0.0:
113         return f"{clean_number(imag, digits)}i"
114
115     sign = "+" if imag >= 0 else "-"
116     return f"{clean_number(real, digits)} {sign} {clean_number(abs(imag), digits)}i"
117
118
119 def print_real_vector(title, values):
120     print(title)
121     print(f"{'j':>3} {'value':>18}")
122     for j, value in enumerate(values):
123         number = value.real if isinstance(value, complex) else value
124         print(f"{'j':>3} {clean_number(number):>18}")
125     print()
126
127
128 def print_complex_vector(title, values, index_name="k"):
129     print(title)
130     print(f"{'index_name':>3} {'value':>28}")
131     for index, value in enumerate(values):
132         print(f"{'index':>3} {format_complex(value):>28}")
133     print()

```

```

134
135
136 def print_verification(title , original , restored , index_name="j",
    original_name="original", restored_name="restored"):
137     print(title)
138     print(f"{index_name:>3} {original_name:>14} {restored_name:>28} {'abs
error':>14}")
139     max_error = 0.0
140     for j, (source , value) in enumerate(zip(original , restored)):
141         error = abs(source - value)
142         max_error = max(max_error , error)
143         print(
144             f"{j:>3} "
145             f"{format_complex(source):>14} "
146             f"{format_complex(value):>28} "
147             f"{error:>14.3e}"
148         )
149     print(f"max error = {max_error:.3e}")
150     print()
151
152
153 def main():
154     variant , values = parse_variant(VARIANT_FILE)
155
156     print("=" * 78)
157     print("LAB 5: DISCRETE FOURIER TRANSFORM")
158     print("=" * 78)
159     print(f"Variant: {variant}")
160     print(f"Number of samples: n = {len(values)}")
161     print()
162
163     print_real_vector("Initial signal y(j):", values)
164
165     print("-" * 78)
166     print("TASK 1. Direct DFT and inverse DFT")
167     print("-" * 78)
168     amplitudes = dft(values)
169     restored = inverse_dft(amplitudes)
170
171     print_complex_vector("DFT amplitudes F(k):", amplitudes)
172     print_complex_vector("Inverse DFT result y(j):", restored , index_name="j")
173     print_verification("DFT reconstruction check:", values , restored)
174
175     print("-" * 78)
176     print("TASK 2. Fast Fourier transform by time decimation")
177     print("-" * 78)
178     (
179         even_values ,

```

```

180         odd_values ,
181         even_amplitudes ,
182         odd_amplitudes ,
183         fft_amplitudes ,
184     ) = fft_decimation_by_time(values)
185
186     print_real_vector("Even-index samples  $y^0(j) = y(2j):$ ", even_values)
187     print_real_vector("Odd-index samples  $y^1(j) = y(2j + 1):$ ", odd_values)
188     print_complex_vector("DFT by even samples  $F^0(k):$ ", even_amplitudes)
189     print_complex_vector("DFT by odd samples  $F^1(k):$ ", odd_amplitudes)
190     print_complex_vector("FFT amplitudes  $F(k):$ ", fft_amplitudes)
191
192     y0_inverse , y1_inverse , fft_restored = inverse_fft_decimation_by_time(
193         fft_amplitudes)
194
195     print_complex_vector("Inverse FFT auxiliary vector  $Y^0(j):$ ", y0_inverse ,
196         index_name="j")
197     print_complex_vector("Inverse FFT auxiliary vector  $Y^1(j):$ ", y1_inverse ,
198         index_name="j")
199     print_complex_vector("Restored signal from inverse FFT  $y(j):$ ",
200         fft_restored , index_name="j")
201
202     print_verification(
203         "FFT amplitudes compared with direct DFT:",
204         amplitudes ,
205         fft_amplitudes ,
206         index_name="k",
207         original_name="direct DFT",
208         restored_name="FFT",
209     )
210     print_verification("FFT reconstruction check:", values , fft_restored)
211
212     print("==" * 78)
213     print("DONE")
214     print("==" * 78)
215
216 if __name__ == "__main__":
217     main()

```